

HDOC-17

QUESTION:

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer.

- Generate the largest possible number using its digits
- Generate the smallest possible number using its digits.
- Swap its first and last digits.
- Check whether it is a palindrome number.

Task1: Algorithm:

Algorithm

Step 1: Start.

Step 2: Declare choice as an integer.

Step 3: Display the menu:

1. Generate the largest possible number using its digits.
2. Generate the smallest possible number using its digits.
3. Swap the first and last digits.
4. Check whether the number is a palindrome.

Step 4: Read choice.

Step 5: Use switch(choice):

- Case 1: Call largestNumber().
- Case 2: Call smallestNumber().
- Case 3: Call swapFirstLast().
- Case 4: Call checkPalindrome().
- Default: Display "Invalid choice".

Step 6: Stop.

Algorithm for largestNumber()

1. Input a positive integer n.
2. Create an array freq[10] initialized to 0.
3. Extract each digit using $n \% 10$ and count its frequency.
4. Starting from digit 9 down to 0, display every digit according to its frequency.
5. The displayed digits form the largest possible number.

Algorithm for smallestNumber()

1. Input a positive integer n.
2. Count the frequency of each digit from 0 to 9.
3. Find and display the smallest non-zero digit first.
4. Display all zeros after it.
5. Display the remaining digits in increasing order.
6. The displayed digits form the smallest possible number.

The non-zero digit is placed first because an integer cannot begin with 0.

Algorithm for swapFirstLast()

1. Input a positive integer n.
2. Find the last digit using $n \% 10$.

3. Find the first digit by repeatedly dividing a temporary copy of n by 10.
4. Find the positional value of the first digit.
5. Keep the middle digits unchanged.
6. Place the last digit at the beginning and the first digit at the end.
7. Display the new number.

Algorithm for checkPalindrome()

1. Input a positive integer n.
2. Store n in temp.
3. Initialize reverse = 0.
4. Extract each digit from temp.
5. Generate the reversed number using:
reverse = reverse * 10 + digit
6. Compare reverse with the original number.
7. If both are equal, display "Palindrome number".
8. Otherwise, display "Not a palindrome number".

Task2: Test case table:

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	57291	Largest possible number = 97521	Largest possible number = 97521	PASS
2	2	31042	Smallest possible number = 10234	Smallest possible number = 10234	PASS
3	3	9876	Number after swapping = 6879	Number after swapping = 6879	PASS
4	4	12321	Palindrome number	Palindrome number	PASS
5	4	12345	Not a palindrome number	Not a palindrome number	PASS

Task3: C program:

```
#include <stdio.h>
void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void checkPalindrome(void);
int main()
{
    int choice;
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
```

```

switch(choice)
{
    case 1:
        largestNumber();
        break;
    case 2:
        smallestNumber();
        break;
    case 3:
        swapFirstLast();
        break;
    case 4:
        checkPalindrome();
        break;

default:
    printf("Invalid choice\n");
}
return 0;
}
/* Function to generate largest possible number */
void largestNumber(void)
{
    int n, digit, freq[10] = {0};
    int i;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    while(n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
    }
    printf("Largest possible number = ");
    for(i = 9; i >= 0; i--)
    {
        while(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
        }
    }
    printf("\n");
}
/* Function to generate smallest possible number */
void smallestNumber(void)
{
    int n, digit, freq[10] = {0};
    int i;

```

```

printf("Enter a positive integer: ");
scanf("%d", &n);
while(n > 0)
{
    digit = n % 10;
    freq[digit]++;
    n = n / 10;
}
printf("Smallest possible number = ");
/* Print smallest non-zero digit first */
for(i = 1; i <= 9; i++)
{
    if(freq[i] > 0)
    {
        printf("%d", i);
        freq[i]--;
        break;
    }
}
/* Print zeros */
while(freq[0] > 0)
{
    printf("0");
    freq[0]--;
}
/* Print remaining digits */
for(i = 1; i <= 9; i++)
{
    while(freq[i] > 0)
    {
        printf("%d", i);
        freq[i]--;
    }
}
printf("\n");
}
/* Function to swap first and last digits */
void swapFirstLast(void)
{
    int n, temp;
    int first, last;
    int power = 1;
    int middle, result;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if(n < 10)
    {
        printf("Number after swapping = %d\n", n);
        return;
    }
}

```

```

last = n % 10;
temp = n;
while(temp >= 10)
{
    temp = temp / 10;
    power = power * 10;
}
first = temp;
middle = (n % power) / 10;
result = last * power + middle * 10 + first;
printf("Number after swapping = %d\n", result);
}
/* Function to check palindrome */
void checkPalindrome(void)
{
    int n, temp;
    int digit;
    int reverse = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        reverse = reverse * 10 + digit;
        temp = temp / 10;
    }
    if(reverse == n)
    {
        printf("Palindrome number\n");
    }
    else
    {
        printf("Not a palindrome number\n");
    }
}

```

Task4: Output:

```
(kali㉿kali)-[~/hdoc17.c]
└─$ ./a.out
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice : 1
Enter a positive integer: 57291
Largest possible number = 97521
```

```
(kali㉿kali)-[~/hdoc17.c]
└─$ ./a.out
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice : 2
Enter a positive interger: 31042
Smallest possible umber = 10234
```

```
(kali㉿kali)-[~/hdoc17.c]
└─$ ./a.out
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice : 3
Enter a positive interger: 9876
Number after swapping = 6879
```

```
(kali㉿kali)-[~/hdoc17.c]
└─$ ./a.out
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice : 4
Enter a positive integer: 12321
Palindrome number
```

```
(kali㉿kali)-[~/hdoc17.c]
└─$ ./a.out
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice : 4
Enter a positive integer: 12345
Not a palindrome number
```